

CredChain: An Empirical Evaluation of On-Chain and Off-Chain Credential Storage

Rea Tasneem Ara
Tandon School of Engineering
New York University
tra9226@nyu.edu

Zhixiong Chen^{*†}
^{*}Tandon School of Engineering
New York University, [†]Mercy University
zxchen@ieee.org

Paul M. Torrens
Tandon School of Engineering
New York University
torrens@nyu.edu

Abstract—We introduce a solution to concerns about placing academic record and credential data directly on-chain: we propose and demonstrate a hash-anchored credential registry on an Ethereum-compatible network as a proof of concept and investigate application scenarios for academic diploma and GPA records. Our approach makes use of only a `keccak256` anchor on-chain, allowing the credential record to remain off-chain, with positive implications for privacy preservation of individuals’ personal details. We show an operational prototype, stored locally, in which the storage layer can be replaced with an IPFS content identifier or another decentralized storage mechanism such as Web3 in future iterations. We describe an empirical assessment of the storage trade-off, combining the gas cost, monetary cost, and privacy exposure of hash anchoring (versus storing the full record on-chain). Our analysis shows that hash anchoring can reduce issuance gas by around 46%, while simultaneously avoiding the storage of plain-text credential attributes on the public ledger. Our approach raises important operational factors to identify the conditions under which the blockchain is and is not an appropriate substrate for credential verification.

Index Terms—blockchain, academic credentials, `keccak256`, hash anchoring, smart contracts, privacy, Ethereum, Sepolia

I. PROBLEM DEFINITION AND BLOCKCHAIN FIT

A. Problem Statement

Credentials, such as academic qualifications, hold lifelong value, and they are vulnerable to manipulation attempts that, when widespread, erode their value to the holder and the issuer. Surprisingly, there is no centralized agency or standard for authenticating academic credentials, leaving the education community with four significant vulnerabilities. First, *fraud and forgery* are easy, cheap, and pervasive [1], [2]. Second, verification is *operationally inefficient* involving a manual, slow and costly look-ups [3]. Third, centralized institutional databases present a *single failure surface* that is straightforwardly vulnerable across many axes: compromise, data loss, and insider manipulation [1], [4]. Fourth, graduates have *little autonomy* in navigating this system despite academic credentials being lifelong tokens of accomplishment leading to inhibiting the mobility of paper credentials [2], [3]. Blockchain credential verification could heal some of these

flaws by providing a verification substrate that is tamper-evident, independently checkable, and resilient to the issuer’s continued existence. Here, we aim to provide the literary testing, examining computational costs while considering on-chain and off-chain solutions to mitigate privacy concerns.

B. User Roles

We model the system as a decentralized ecosystem of four actors. First, *Students/Graduates (Holders)* fulfill academic requirements, use an Ethereum account, and are associated with the issued credential. They control disclosure, presenting a proof or granting a verifier access without necessarily exposing the raw record [1], [5]. Second, *Higher-Education Institutions (Issuers)* verify student identity, detail academic outcomes, sign the resulting e-certificate, and manage revocation in cases of error or misconduct [2], [5], [6]. Third, *Verifiers (Third Parties)*, as employers, recruiters, or other interested parties, query the ledger to authenticate a presented credential. They perform this query instantly, without an intermediary and without direct contact with the issuer [2]. Fourth, *Governing Bodies (Registrar/Regulator)* provide an accreditation root, attesting which institutions are authorized to issue academic credentials or licenses [4]. In our prototype, this role is implemented as the contract administrator, which allows issuers to anchor the chain of trust.

C. What This Paper Targets

Blockchain is suited to this problem for several technical reasons. *Immutability* makes a recorded credential anchor permanent and tamper-evident [2], [3]. A shared blockchain registry reduces reliance on each issuer’s internal verification database and preserves the on-chain issuance and revocation history, even if the issuer’s original system becomes unavailable [1], [6]. *Transparency and traceability* give every issuance, signature, and revocation a timestamped, and auditable trail back to an authorized issuer [3], [7]. *Cryptographic security*, through hashing and digital signatures, makes any post-issuance alteration immediately detectable [4]. *Smart contracts* automate verification logic, compressing a multi-day process into seconds [1].

Credential Verification Workflow

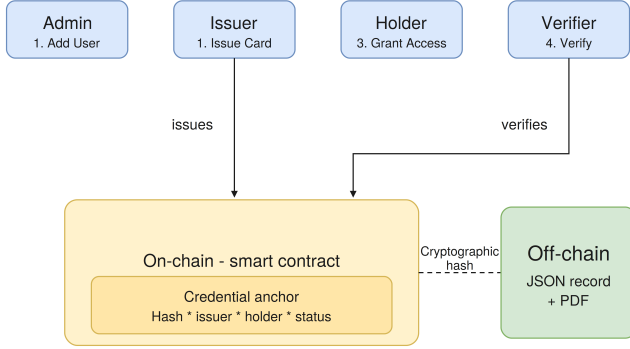


Fig. 1. The administrator authorizes an issuer (1); the issuer anchors a credential on-chain (2); the holder grants a verifier access to the record (3); and the verifier checks on-chain status and rehashes the locally stored off-chain record to confirm its integrity (4). Only a `keccak256` hash anchor and minimal status metadata are stored on-chain, while the full record remains off-chain.

Prior work largely surveys blockchain’s suitability for credentialing [6]. Production efforts have already standardized how such credentials are expressed and exchanged such as the W3C Verifiable Credentials data model, Blockcerts, and EBSI [8]–[10], yet do not isolate or quantify the gas, cost, and privacy consequences of where the credential records live. This paper measures the gas, cost, and privacy tradeoff between anchoring credentials by their hash and storing the full credential record on-chain based on four roles described under the *User Roles* section.

This paper includes a Solidity proof-of-concept with a reproducible test suite covering the full lifecycle and adversarial cases, an empirical evaluation, and a scoped discussion of limitations and future work across the Systems Design, Security and Privacy sections. Additionally, we note that all reported measurements use locally stored off-chain records so that the evaluation isolates the blockchain storage tradeoff. Decentralized-storage upload, retrieval, availability, and latency are outside the scope of the present evaluation. GitHub Repository for implementation can be found here: github.com/tra9226/appliedblockchain_summer2026.

II. SYSTEM DESIGN AND WORKFLOW

A. Architecture

The system is a three-layer design (Fig. 2). At the top, four actors hold Ethereum accounts and interact with the registry through their wallets. The middle layer is a single smart contract, `CredentialRegistry`, deployed on an Ethereum-compatible network. The contract holds all trust state, including which issuers are authorized, the credential anchors, and the holder-granted access flags.

The bottom layer is off-chain and comprises the full credential record in JSON format and its PDF rendering, neither of which is written to the blockchain. The two layers are

End to End System Architecture

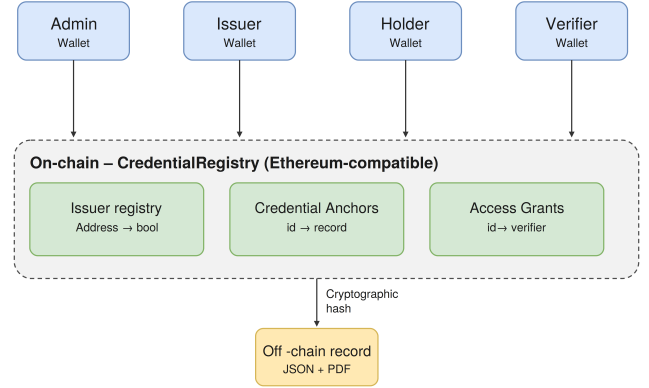


Fig. 2. Four actor wallets admin, issuer, holder, and verifier interact with the `CredentialRegistry` smart contract deployed on an Ethereum-compatible network. Each on-chain anchor is bound to its off-chain JSON record and PDF by a `keccak256` hash, so authenticity and status are verifiable on-chain while personal data remains off the ledger.

bound by a `keccak256` commitment computed from the standardized JSON record, so that any later modification can be detected by rehashing and comparing the result with the on-chain anchor. In the current prototype, the JSON record and PDF are stored locally. The local storage reference can later be replaced with an IPFS CID or another decentralized storage reference without changing the core anchoring and verification logic.

B. Workflow

The credential lifecycle (Fig. 1) runs in five steps. First, the administrator authorizes a university by recording its address in the issuer registry. Second, the issuer computes $id = \text{keccak256}(\text{record})$ off-chain and calls `issueCredential(id, holder, metadataURI)`, writing the anchor and emitting a `CredentialIssued` event. Third, when a student or graduate wants to present the credential, the holder calls `grantAccess(id, verifier)` to authorize a specific employer. Fourth, the verifier calls `verifyCredential(id)` to read the issuer, holder, status, and timestamps, then rehashes the off-chain file it received and confirms that the hash matches the anchor. Fifth, if a credential is issued in error or rescinded, the issuing university calls `revokeCredential(id)`, flipping the status flag. In this case, verifiers see the change immediately on their next lookup.

C. Data model

The off-chain record is a versioned JSON document (schema `credchain.diploma.v1`) containing the issuer block (name, DID, wallet), the holder block (name, student ID, wallet), the credential block (program, honors, GPA, conferral date, transcript digest), and a random nonce. The nonce matters. Without it, a record built only from public fields could be hash-guessed by an adversary, so it adds entropy to

TABLE I
ON-CHAIN FIELDS STORED PER CREDENTIAL

Field	Type	Purpose
issuer	address	which authorized university issued it
holder	address	the owning student
issuedAt/updatedAt	uint64	issuance / last-status-change time
status	enum	{None, Active, Revoked}
metadataURI	string	Off-chain record reference (local/CID)

the commitment. The on-chain counterpart stores the anchor and metadata needed to establish issuer authorization, holder association, and credential status, as summarized in Table I.

The contract keys credentials by the `bytes32` id in a `credentials` mapping, tracks authorized issuers in `isIssuer/issuerName`, and holds disclosure flags in a nested `accessGranted[id][verifier]` mapping.

D. Role permissions.

Authorization is enforced by modifiers (`onlyAdmin`, `onlyIssuer`) and explicit ownership checks inside each function (Table II) where the asymmetry is deliberate. *Authenticity and Status are public* (anyone can verify that a credential exists, was issued by an authorized institution, and remains active), but the contract restricts retrieval of the record reference through its application interface to the holder, issuer, or an authorized verifier.

TABLE II
ROLE PERMISSIONS

Function	Admin	Issuer	Holder	Verifier
add/removeIssuer	✓	NA	NA	NA
issueCredential	NA	✓ ^a	NA	NA
revokeCredential	NA	✓ ^b	NA	NA
grant/revokeAccess	NA	NA	✓	NA
getMetadataURI	NA	✓ ^b	✓	✓ ^c
verify/isValid	public (anyone)			

^aif authorized ^bissuing issuer only ^cif granted

A checkmark denotes a permitted call, with conditions given in the footnotes; NA denotes a call the role cannot make.

E. Design boundary

On-chain, the system handles issuer authorization, hash anchoring, the active/revoked credential lifecycle, holder-controlled authorization flags, and a tamper-evident event log. Off-chain, it handles local storage of the credential JSON and PDF, canonical hashing, and client-side verification by rehashing. Some additional features could include zero-knowledge selective disclosure [11], DID method resolution [12], IPFS storage and availability guarantees [13], regulator multi-signature accreditation, encryption and key distribution, and key-loss recovery. These are out of scope for this paper but we acknowledge the development potential. The on-chain access flag records authorization intent, but the off-chain application must enforce access to the record, and the blockchain cannot retract a file that a verifier has already received.

III. SMART CONTRACT IMPLEMENTATION

A. Issuer management

Only the administrator can change the set of authorized issuers. To achieve this, `addIssuer` rejects the zero address, requires a non-empty institution name, and refuses to re-add an existing issuer. Meanwhile, `removeIssuer` clears both the flag and the stored name. Each action emits an event so the issuer registry has a complete, auditable history.

B. Credential lifecycle

Issuance is restricted to authorized issuers via `onlyIssuer`. The function validates inputs and, critically, requires `status == None` so that a credential ID can never be silently overwritten. This guards against duplicate or replay issuance. We further restrict revocation. The *issuing* issuer (not just any issuer) is enforced by the `c.issuer == msg.sender` check. Status is stored as an enum, and timestamps are recorded for the audit trail.

C. Holder-controlled access

The holder controls disclosure authorization through `grantAccess` and `revokeAccess`. Both functions require the caller to be the credential holder, ensuring that only the student/graduate can authorize a verifier. (And this returns some agency to the credential holder.) In the current prototype, the application checks this authorization before providing access to the locally stored credential record. The on-chain access flag records and enforces application-level authorization, but it does not by itself make an on-chain storage value confidential. A future implementation could use the same authorization mechanism with encrypted, CID-addressed storage, and controlled key distribution.

D. Verification views and events

Public verification needs no permissions: `verifyCredential` returns whether the credential exists, who issued it, who holds it, its status, and its timestamps, while `isValid` is a convenience boolean; and `hasAccess` lets a verifier check its own authorization. Because these are view functions, they incur no blockchain transaction fee when called through an RPC interface. The contract emits six events for every state change (issuer added/removed, credential issued/revoked, access granted/revoked), providing an external audit log for external indexer and block explorer.

IV. DATA, STORAGE, AND VERIFICATION

A. Off-Chain Storage and Retrieval

Again, we mention that the credential JSON and PDF are stored locally *outside* the blockchain. The `metadataURI` field serves as the application-level reference to the off-chain record, and the application checks the holder's authorization before releasing it to a verifier. Keeping the full record off-chain avoids placing plaintext academic attributes on the public ledger.

The local storage reference can later be replaced with an IPFS CID or another decentralized storage reference without

changing the core smart contract anchoring logic. Encryption, CID-based retrieval, pinning, availability, and key distribution are not implemented or evaluated in the present prototype and remain targets for future work.

B. Credential Integrity and Verification

The issuer computes a `keccak256` hash of the canonical credential record and stores the resulting anchor on-chain. During verification, the verifier loads the local record, applies the same standardization procedure, recomputes the hash, and compares it with the on-chain anchor. A match confirms that the record has not changed since issuance. The local storage layer can later be replaced by IPFS-backed storage, with the returned CID serving as the pointer to the stored object. In that extension, the CID would protect the integrity of the stored object, while the on-chain `keccak256` anchor would continue to protect the integrity of the underlying credential record. Implementing and evaluating decentralized storage would provide promising avenues for future work.

C. Issuer authenticity via transaction signatures

CredChain does not implement a separate application-layer signature scheme over the credential record. Instead, it relies on the digital signature intrinsic to every Ethereum transaction. Each call to `issueCredential` is a state-changing transaction the issuing institution must sign with its private key, and the EVM verifies this ECDSA signature over the `secp256k1` curve before the call is admitted. The contract reads the recovered signer as `msg.sender` and gates issuance through `onlyIssuer`, which requires `isIssuer[msg.sender]` to be true. Because that flag can only be set by the administrator through `addIssuer`, the anchoring of a credential is cryptographically bound to a specific, administrator-authorized institutional key. As a consequence, an adversary cannot forge an issuance without the corresponding private key, and an unauthorized key is rejected even if the signature is valid. Indeed, this property is directly exercised by the “unauthorized account cannot issue (fake issuer threat)” test.

D. Design rationale

A common alternative is to have the issuer sign the credential payload itself (e.g., an EIP-712 typed-data signature recovered on-chain via `ecrecover`). We judged this as redundant for our threat model. The two guarantees such a scheme would provide are already supplied: the *integrity* of the off-chain record is established by the `keccak256` anchor (any modification changes the credential ID and breaks the match), and the *authenticity* of the issuing party is established by the transaction signature and allow-list check. Together, these bind an authorized issuer to one specific, tamper-evident record hash. This is precisely the assurance a verifier needs. We note the boundary of this choice: the transaction signature authenticates the *act of anchoring* a given credential ID, not a standalone signature that can be checked offline without consulting the chain. For a registry whose canonical source of truth is the on-chain anchor, that boundary is acceptable. A

system requiring portable, offline-verifiable credentials would add payload-level signatures, which we identify as a next step to pursue.

V. TESTING AND REPRODUCIBILITY

The implementation is validated by an automated suite of 14 Mocha/Chai tests executed with Hardhat against a local EVM, all passing. The suite is organized around the trust model rather than around functions, so each adversarial case maps to a specific security property. Issuer-management tests confirm that the administrator can authorize and remove issuers, whereas a non-administrator cannot. Issuance tests confirm that an authorized issuer can issue, that an *unauthorized* account cannot (the fake-issuer threat), and that the same credential ID cannot be issued twice (replay/duplicate). Revocation tests confirm that the issuing issuer can revoke, that a *different* issuer cannot revoke someone else’s credential, and that an already-revoked credential cannot be revoked again.

Access-control tests confirm the holder-controlled authorization flow: a verifier without access is denied access to the metadata reference via the contract interface. The holder can grant access, and after this the verifier can retrieve it, but again only the holder can grant that access (and may also revoke it). A final test confirms that verification of an unknown ID returns `exists = false`. Each negative test demonstrates a mitigation rather than assuming it, so the security analysis in Section VII is backed by executable evidence.

The off-chain → on-chain link is independently reproducible. Running `node offchain/hashRecord.js` recomputes the credential identifier as `keccak256` from the standardized sample record, yielding `0x859b3956...967269`, which is exactly the ID an issuer passes to `issueCredential` and a verifier recomputes to confirm a match. Beyond local testing, the contract was deployed to the public Ethereum *Sepolia* testnet (chainId 11155111) at address `0xC45b2a1cDd177Fb0b53F51066522648a94c96c9D`, and the complete lifecycle (authorize issuer, issue, deny an unauthorized verifier, grant access, read, and revoke) was exercised end-to-end through a browser front end with four distinct wallets, one per role. Every state-changing call is a real, timestamped transaction, and the emitted `CredentialIssued`, `AccessGranted`, and `CredentialRevoked` events are visible on the public block explorer, confirming that the system behaves identically on a real network. Full reproduction steps, from a clean clone through compilation, tests, the hash check, local-node demo, and the Sepolia deployment, are documented in the project README on our repo. The test suite and Sepolia demonstration use locally stored off-chain records; CID-based decentralized storage is not implemented or tested in the present prototype.

VI. EVALUATION AND TRADE-OFF DISCUSSION

The central design decision in CredChain is where the credential record lives: either anchored off-chain by hash or stored in full on-chain. To evaluate this empirically rather than assert

it, we implemented both designs, CredentialRegistry (hash-anchored) and NaiveCredentialRegistry (full record on-chain), and measured the same operation, issuance of the identical Jane credential, on each. The baseline is feature identical to the anchored contract with the same issuer authorization, status lifecycle, timestamps, events, and holder-controlled access. The two differ in storage strategy where the baseline writes the full record on chain whereas the anchored design stores only the `keccak256` hash. The measurement harness deploys both contracts, issues the same record on a local Hardhat node, and reports the real `gasUsed` values from the transaction receipts and the captured output is committed for reproducibility. The credential ID is recomputed from the off-chain JSON at runtime, so both designs anchor provably the same record. The present evaluation measures the blockchain storage trade-off using locally stored off-chain records. It does not evaluate decentralized storage upload costs, retrieval latency, or availability.

A. Gas and monetary cost

Issuing one credential costs 162,271 gas under the hash-anchored design versus 303,298 gas under the full-record baseline, a reduction of 141,027 gas, or approximately 46.5%, a promising result. At illustrative network conditions of 30 gwei and an ETH price of \$2,500, these values correspond to approximately \$12.17 and \$22.75 per issuance, respectively. These monetary values are illustrative, and gas usage is the primary comparison. Because on-chain verification uses only view calls that incur no transaction fee, this one-time issuance cost stands in for the recurring manual-verification fee noted earlier: at an illustrative \$3.48 per manual check, the \$12.17 anchored issuance is recovered after roughly four verifications, and every verification thereafter is effectively free, making the lifetime cost explicit rather than assumed. The gas counts are, moreover, independent of the execution layer even though the dollar figures are not—the same 162,271 gas settles for a substantially smaller fee on Ethereum layer-2 rollups than at the mainnet conditions assumed here, which would push the break-even lower still. We do not measure layer-2 fees directly and leave that validation to future work. Deployment gas is a controlled comparison because the contracts are feature-identical except for record storage. The anchored design uses 1,133,614 gas to deploy versus 1,279,859 gas for the full-record baseline, an 11.4% reduction in one-time deployment gas.

B. Scalability

By design, hash-anchored issuance stores a fixed-size 32-byte digest, regardless of the credential record’s size, whereas full-record storage grows with the credential’s size. The evaluated sample record showed an approximately 46.5% reduction in issuance gas. Across transcripts ranging from one to forty courses, anchored issuance stayed constant at 162,271 gas while the full-record baseline grew linearly, at roughly 45,400 gas per course, to 2,119,053 gas. At a full transcript, the full-

record baseline consumes roughly $13\times$ the gas of the anchored design ($13.06\times$).

C. Privacy and security

The full-record baseline writes the student/graduate’s name, program, GPA, and dates to a public and permanent ledger. The hash-anchored design avoids placing these plaintext academic attributes on-chain and instead stores a `keccak256` commitment. Any modification to the off-chain record changes the recomputed hash, breaking the match with the on-chain anchor. The random nonce included in the credential record also reduces the risk of guessing the contents from predictable public fields. Both designs bind issuance to an authorized institutional transaction signer, but the anchored design provides this verification without publishing the full academic record. Wallet addresses, timestamps, and credential status remain visible and may still create linkability.

D. Latency, UX, and complexity

Both designs issue in a single transaction with no measurable latency difference on a local node, and both incur one block’s confirmation on a public network. The issuer must store the record and serve it to authorized verifiers. The verifier must recompute-and-match rather than read fields directly from the chain, in exchange for the gas, scalability, and privacy gains above. The anchored design adds modest local file management complexity because the issuer must retain the record, and the verifier must retrieve, standardize and rehash it rather than reading its attributes directly from the blockchain.

Overall, the measured hash-anchored design reduces issuance gas by approximately 46.5% and avoids storing plaintext academic attributes on the public ledger. Its fixed-size commitment also separates on-chain storage requirements from credential-record size, as our record-size sweep from one to forty courses confirms: on-chain issuance cost stays constant while the full-record baseline grows linearly. Under the evaluated assumptions, hash anchoring provides a preferable cost-privacy trade-off for academic credential verification, at the expense of requiring an external storage and retrieval mechanism.

VII. SECURITY AND PRIVACY REASONING

CredChain’s threat model centers on five realistic attacks, each closed by a specific mechanism and verified by an adversarial test. The issuer allowlist prevents a forged credential from a fake issuer: `issueCredential` is gated by `onlyIssuer`, and only the admin can authorize issuers, so an unauthorized key is rejected even with a valid signature. Replay or duplicate issuance is blocked by the `Status.None` precondition, which rejects any id that has already been issued. Cross-issuer tampering, in which one institution revokes another’s credential, is prevented by the `c.issuer == msg.sender` check in `revokeCredential`.

Unauthorized retrieval through the contract interface is mitigated by holder-controlled access where the metadata reference is returned only to the holder, the issuing institution, or

a verifier explicitly authorized by the holder. This mechanism enforces application level authorization, but it does not render values stored on a public blockchain confidential.

Spoofed verification of a non-existent record is handled by the default `Status.None` state, so unknown IDs return `exists = false`. Each is exercised by a corresponding negative test, so that the guarantees are demonstrated rather than assumed.

On privacy, the core mitigation is architectural. No plaintext academic attributes (such as the student's name, degree, GPA, or transcript) are written on-chain. The contract stores a `keccak256` commitment together with issuer and holder addresses, status flags, and timestamps. The commitment provides tamper evidence without directly disclosing the credential contents, although the remaining metadata may create linkability. The random nonce in the off-chain record reduces the feasibility of guessing predictable credential contents from the hash.

A. Limitations and Future Work

We note several limitations. The design assumes an honest administrator. A compromised administrator key could authorize a malicious issuer, so a production implementation should use multi-signature control or a governance contract. Authenticity is enforced through a traceable signature attached directly to the credential, which means verification requires consulting the chain. The current prototype depends on local off-chain storage, which we tout as a benefit. Still, it implies that a verifier can detect a missing or modified record but cannot recover an unavailable record. Moreover, a holder authorization cannot prevent a verifier from retaining or redistributing a record once it is disclosed. Institutional issuer and administrator keys should be held in hardware security modules or split with threshold and multi-signature signing behind a timelocked governance contract, so that no single compromised key can authorize a rogue issuer. The off-chain record should be encrypted under keys released only to authorized verifiers, with the on-chain `keccak256` anchor still guaranteeing integrity, and served from content-addressed storage (IPFS, Web3, CIDs) with persistent pinning for availability. Redistribution by an authorized verifier is a fundamental limit of any disclosed bearer record, and is better bounded by short-lived, watermarked presentations or by zero-knowledge selective disclosure, so that the raw record never leaves the holder, than by on-chain controls. Lost holder keys, in turn, call for social-recovery wallets or issuer-mediated re-binding of a credential to a fresh holder address. We leave the implementation and evaluation of these mechanisms to future work.

VIII. CONCLUSION

CredChain demonstrates that hash anchoring can provide blockchain-based tamper evidence and issuer accountability for academic credentials without exposing plaintext academic attributes on a public ledger. By committing a `keccak256` anchor and minimal status metadata on-chain, the evaluated

design reduces issuance gas by approximately 46.5% relative to the full-record baseline, while keeping the credential record off-chain. Its fixed-size commitment also separates on-chain storage requirements from credential-record size, which our record-size sweep from one to forty courses confirms. Issuer allow-listing and transaction signatures bind each on-chain anchor to an authorized institutional account, while revocation and verification remain publicly auditable. Under the evaluated assumptions, hash anchoring offers a preferable cost-privacy trade-off for academic credential verification. Future work includes replacing local storage with encrypted IPFS-backed storage, evaluating retrieval performance and availability, and integrating decentralized identifiers, controlled key distribution, and multisignature accreditation.

REFERENCES

- [1] A. Rustemi, F. Dalipi, V. Atanasovski, and A. Risteski, "A systematic literature review on blockchain-based systems for academic certificate verification," *IEEE Access*, vol. 11, pp. 64 679–64 802, 2023.
- [2] J.-C. Cheng, N.-Y. Lee, C. Chi, and Y.-H. Chen, "Blockchain and smart contract for digital certificate," in *Proc. IEEE Int. Conf. Applied System Innovation (ICASI)*, Chiba, Japan, April 13-17, 2018, 2018, pp. 1046–1051.
- [3] G. Capece, N. Levialdi Ghiron, and F. Pasquale, "Blockchain technology: Redefining trust for digital certificates," *Sustainability*, vol. 12, no. 21, p. 8952, 2020.
- [4] M. Al Hemairy, M. Abu Talib, A. Khalil, A. Zulfikar, and T. Mohamed, "Blockchain-based framework and platform for validation, authentication and equivalency of academic certification and institution's accreditation: UAE case study and system performance," *Education and Information Technologies*, vol. 29, pp. 18 203–18 232, 2024.
- [5] J. A. Berrios Moya, J. Ayoade, and M. A. Uddin, "A zero-knowledge proof-enabled blockchain-based academic record verification system," *Sensors*, vol. 25, no. 11, p. 3450, 2025.
- [6] A. Rustemi and F. Dalipi, "From concept to practice: A blockchain-based solution for secure and efficient academic credentials in higher education," *IET Blockchain*, vol. 6, no. 1, p. e70030, 2026.
- [7] M. A. Cardenas-Quispe and A. Pacheco, "Ensuring academic integrity with a degree verification prototype," *Scientific Reports*, vol. 15, no. 1, p. 9281, 2025.
- [8] World Wide Web Consortium (W3C), "Verifiable credentials data model v2.0," W3C Recommendation, May 2025, <https://www.w3.org/TR/vc-data-model-2.0/>.
- [9] J. Nazaré, K. Hamilton Duffy, and P. Schmidt, "What we learned from designing an academic certificates system on the blockchain," MIT Media Lab, 2016.
- [10] E. Tan, E. Lerouge, J. Du Caju, and D. Du Seuil, "Verification of education credentials on european blockchain services infrastructure (EBSI): Action research in a cross-border use case between belgium and italy," *Big Data and Cognitive Computing*, vol. 7, no. 2, p. 79, 2023.
- [11] S. Khadka, D. Goswami, and S. Das, "Poster: Privacy-preserving compliance checks on ethereum via selective disclosure," *arXiv preprint arXiv:2602.18539*, 2026.
- [12] S. R. Garzon, D. Natusch, A. Philipp, A. Küpper, H. J. Einsiedler, and D. Schneider, "Did link: Authentication in tls with decentralized identifiers and verifiable credentials," in *2024 21st Annual International Conference on Privacy, Security and Trust (PST)*, Sydney, Australia, August 28-30, 2024. IEEE, 2024, pp. 1–11.
- [13] R. Puneetha, R. Tanuja, and S. Manjula, "Performance evaluation of off-chain storage systems in blockchain environments," in *2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS)*, Bangalore, India, June 28-29, 2024. IEEE, 2024, pp. 1–4.